1 Title of The Project:

TRACE (**T**ask and **R**esource **A**ttendance and **C**omprehensive **E**valuation System)

Introduction:

TRACE is an AI-powered Attendance, Task Management, and Productivity Evaluation System designed for modern organizations. The system uses face recognition, geo-location validation, and smart analytics to monitor employee attendance, manage tasks, and evaluate productivity without requiring biometric hardware devices.

The project helps organizations reduce proxy attendance, improve task tracking, and monitor employee performance in real time. TRACE also includes an AI-based productivity engine that analyzes attendance consistency, task completion, active working hours, and deadline adherence to generate performance reports and productivity scores.

The system aims to provide a secure, scalable, and intelligent workforce management solution for companies, startups, remote teams, and educational institutions.

2 Objective of the Project:

- To develop an AI-powered smart attendance and productivity management system.
- To eliminate proxy attendance using face recognition and geo-location validation.
- To provide real-time task management and employee performance tracking.
- To generate AI-based productivity scores and automated performance reports.
- To reduce manual supervision and improve organizational efficiency.
- To create a secure, scalable, and hardware-independent workforce management solution.

3 Project Category:

The TRACE project primarily falls under the categories of Artificial Intelligence and Machine Learning as it uses AI-based productivity analysis, behavioural monitoring, and face recognition for attendance verification. The project also belongs to Image Processing because it processes facial images for user authentication. It incorporates RDBMS concepts for managing employee, attendance, task, and productivity data using relational databases. Additionally, the system uses OOPS principles in backend development and Web Technology and Cloud Computing for developing a scalable browser-based SaaS platform.

4 Tools, Platform, Hardware and Software Requirements:

4.1 Software Requirements-

Component	Technology	Purpose
Frontend	React.js + Tailwind CSS	User interface — employee & admin dashboards
Backend Framework	FastAPI (Python)	REST API, business logic, auth middleware
ASGI Server	Uvicorn	Production server for FastAPI
Database	PostgreSQL	Persistent relational data storage
ORM	SQLAlchemy + Alembic	DB models and schema migrations
Authentication	JWT (PyJWT)	Stateless secure session tokens
Background Tasks	Celery + Redis	Scheduled report generation, email jobs
Real-time	WebSocket (Socket.io)	Live attendance and task updates
AI / Scoring Engine	Python (scikit-learn, pandas, numpy)	Productivity score, anomaly detection
Email Service	SendGrid / SMTP	Automated weekly report dispatch
Version Control	Git + GitHub	Source control and collaboration
Containerization	Docker + Docker Compose	Consistent deployment environments

4.2 Hardware Requirements-

Requirement	Specification
Developer Machine	Laptop / Desktop with minimum 8 GB RAM, i5 or equivalent
Internet	Broadband connection required (GPS & IP features)
Input Device	Any standard webcam (optional — for face photo capture)
Server (Deployment)	AWS EC2 t3.medium or DigitalOcean Droplet (2 vCPU / 4 GB RAM)

5 Problem Definition and Requirement Specifications:

5.1 Problem Definition

- Proxy attendance: colleagues falsely marking others as present is undetectable without verification.
- Remote employee misreporting: actual working hours are self-declared with zero accountability.
- Absence of productivity measurement: no objective data links hours logged to work delivered.
- Inefficient task follow-up: managers spend significant time tracking task status manually.
- No behavioral insight: repeated delays or idle patterns go unnoticed until performance review.
- No automated reporting: management decisions rely on manually compiled spreadsheets.

5.2 Functional Requirements

ID	Functional Requirement	Actor
FR-01	User registration and login with role assignment (Admin / Manager / Employee)	Admin
FR-02	GPS geo-fencing: mark attendance as Office / Remote / Suspicious based on location radius	Employee
FR-03	IP tracking: flag foreign IPs, rapid IP changes, or multiple users on same IP	System
FR-04	Task creation with title, description, deadline, priority, estimated time, assignee	Manager
FR-05	In-browser task timer: Start / Pause / Complete with active time recording	Employee
FR-06	Productivity Score computation using weighted formula on attendance and task data	AI Engine
FR-07	Behavioral anomaly detection: flag repeated late submissions, long idle periods	AI Engine
FR-08	Weekly AI performance report auto-generated and emailed to admin	Celery / Email
FR-09	Admin analytics dashboard: attendance heatmap, task metrics, employee rankings	Admin
FR-10	Role-based access control: each role sees only permitted modules and data	System

5.3 Technical Specifications

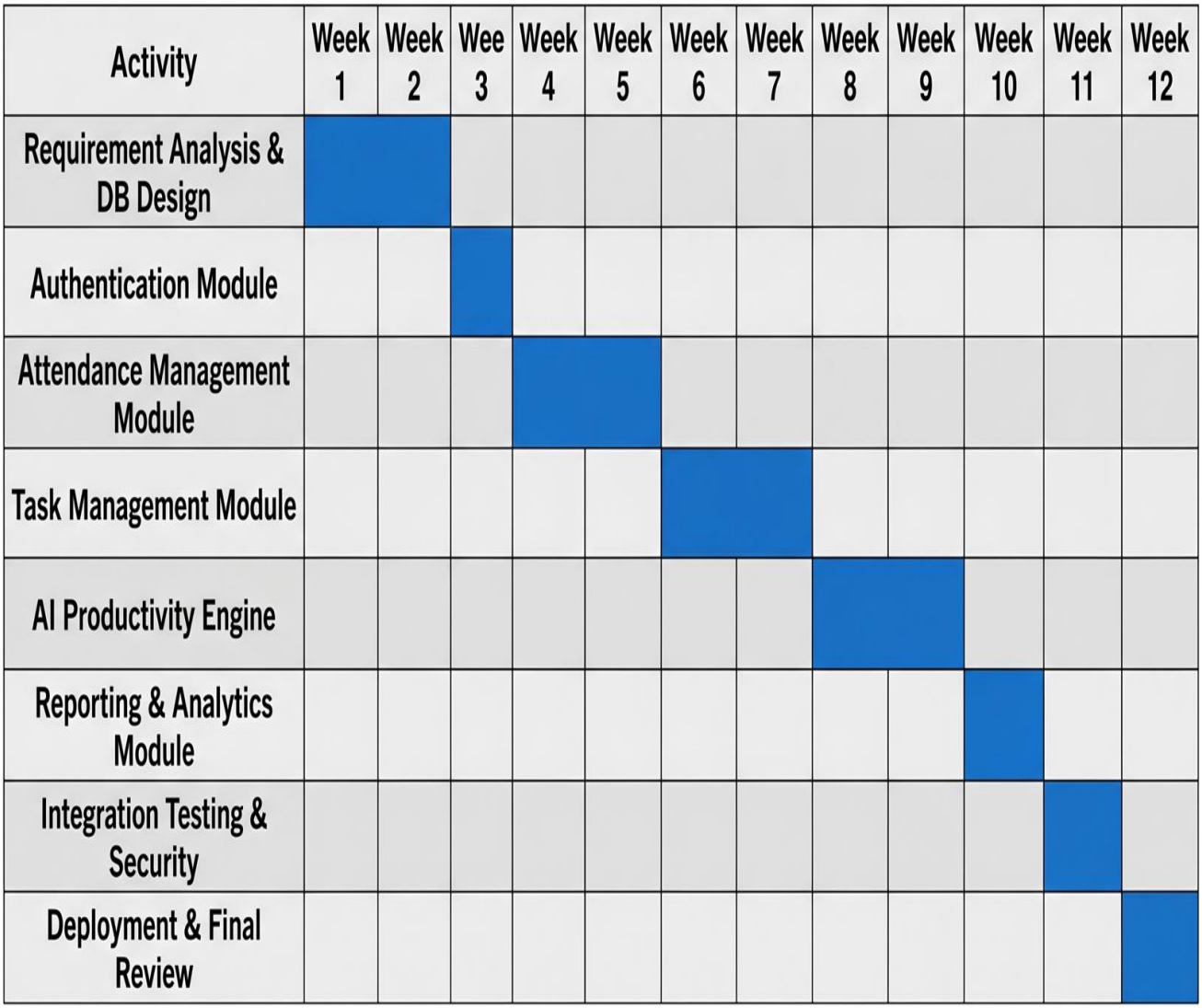
Parameter	Specification
API Response Time	≤ 2 seconds for all standard endpoints
Dashboard Load Time	≤ 4 seconds on standard broadband
Uptime Target	99% (handled via cloud auto-scaling)
Communication Protocol	HTTPS (TLS 1.2+) for all client-server traffic
Authentication	JWT tokens with 8-hour expiry and refresh token support
Database Constraints	Foreign keys, NOT NULL constraints, UNIQUE on email and session fields
Score Computation	Triggered nightly via Celery scheduled task
Report Generation	Every Monday 08:00 AM via Celery beat
Concurrent Users	Supports 200+ concurrent users per deployment instance

5.4 Project Planning and Scheduling

Gnatt Chart

Phase	Activity	Duration
Phase 1	Requirement analysis, wireframes, DB schema design	Week 1–2
Phase 2	Auth module: login, JWT, role-based access, user management	Week 3
Phase 3	Attendance module: geo-fencing, IP tracking, check-in/out	Week 4–5

Phase 4	Task module: CRUD, assignment, timer, priority and deadline tracking	Week 6–7
Phase 5	AI productivity engine: scoring formula, anomaly detection	Week 8–9
Phase 6	Reporting module: weekly email report, PDF export, dashboards	Week 10
Phase 7	Integration testing, bug fixes, security hardening	Week 11
Phase 8	Deployment, Docker setup, cloud configuration, final review	Week 12



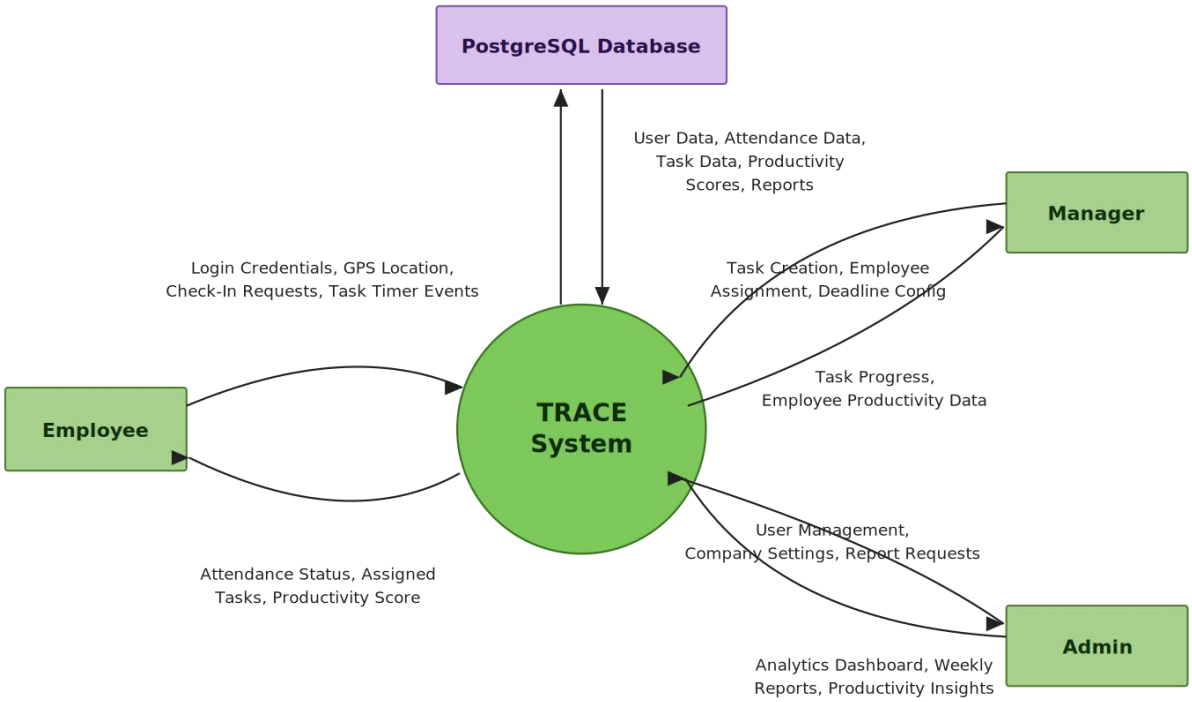
6. Scope of the Solution :

- IT companies and software agencies managing distributed or hybrid teams.
- Startups needing lightweight workforce analytics without dedicated HR software.
- Educational institutions tracking faculty attendance and task submissions.
- Remote-first organizations requiring verifiable work-hour data.
- Multi-company SaaS deployment: each company is a tenant with isolated data and settings.
- Future payroll integration based on verified attendance and productivity scores.

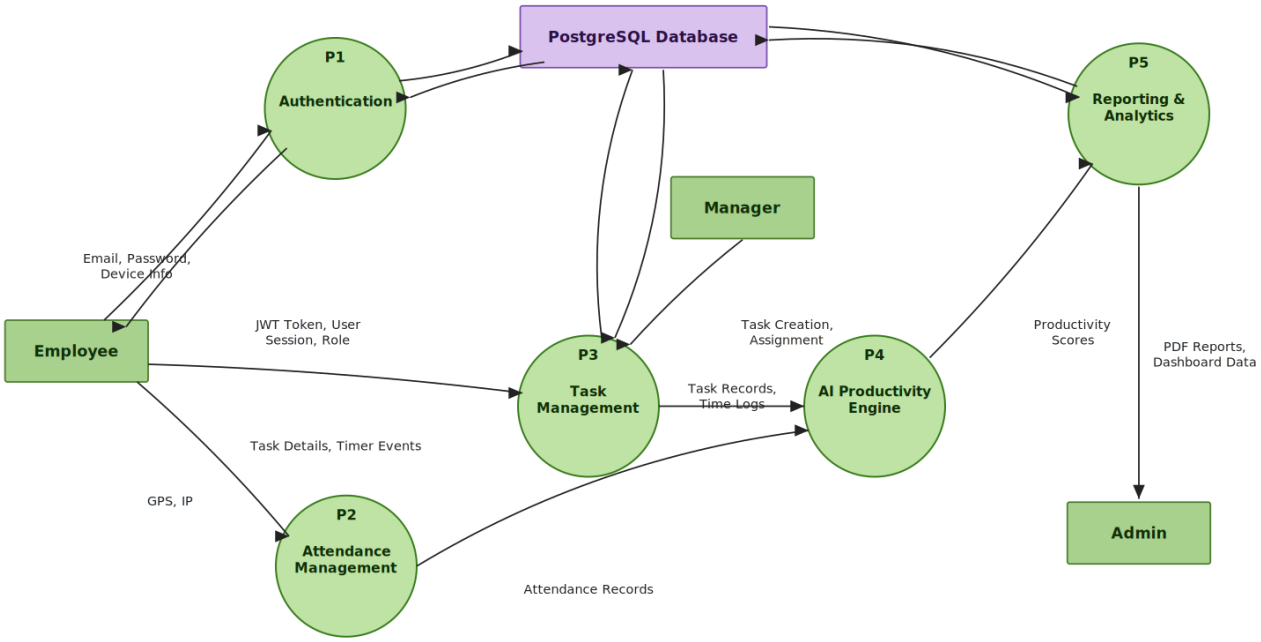
7. Analysis and System Design:

7.1 Data Flow Diagram-

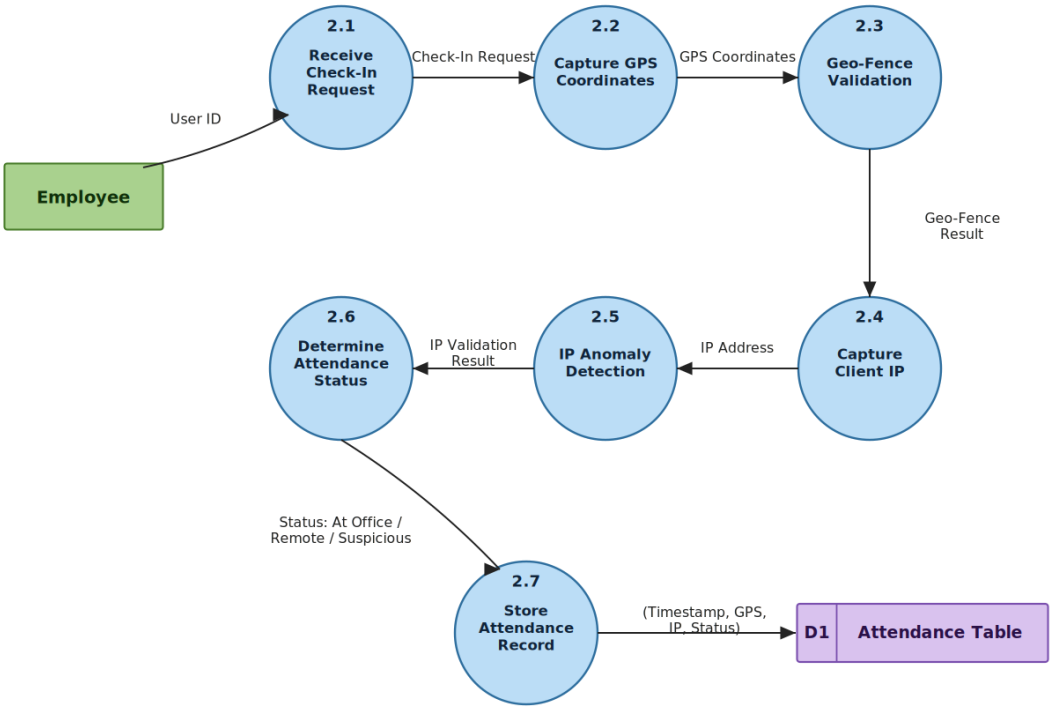
Level 0 DFD-

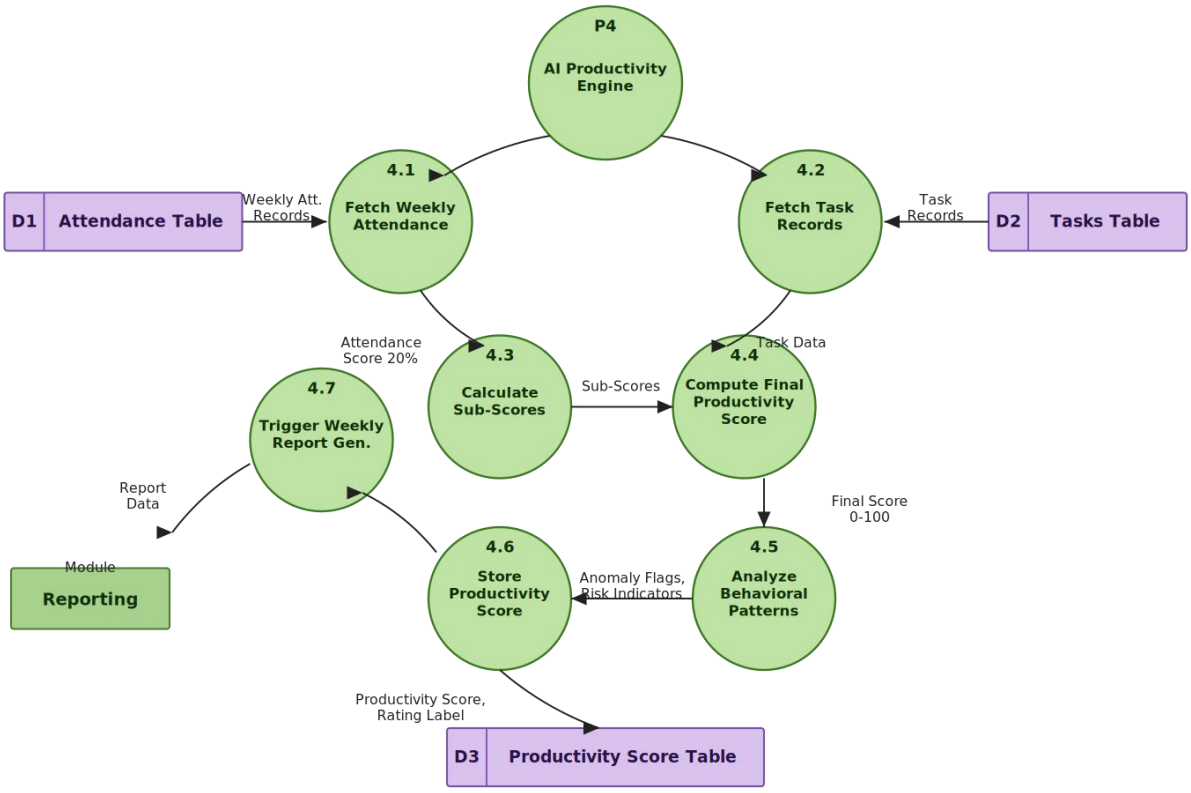


Level 1 DFD-

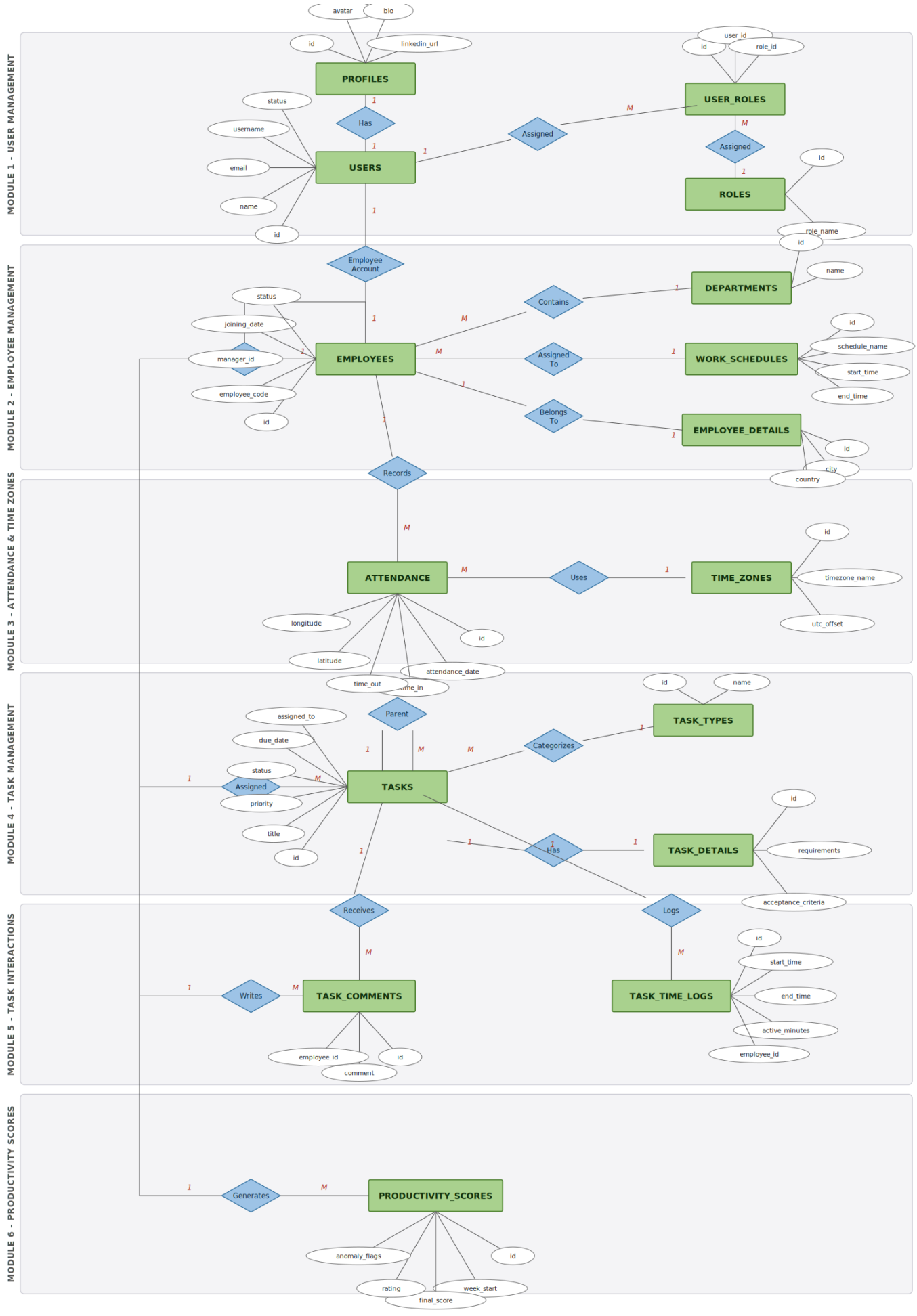


Level 2 DFD-

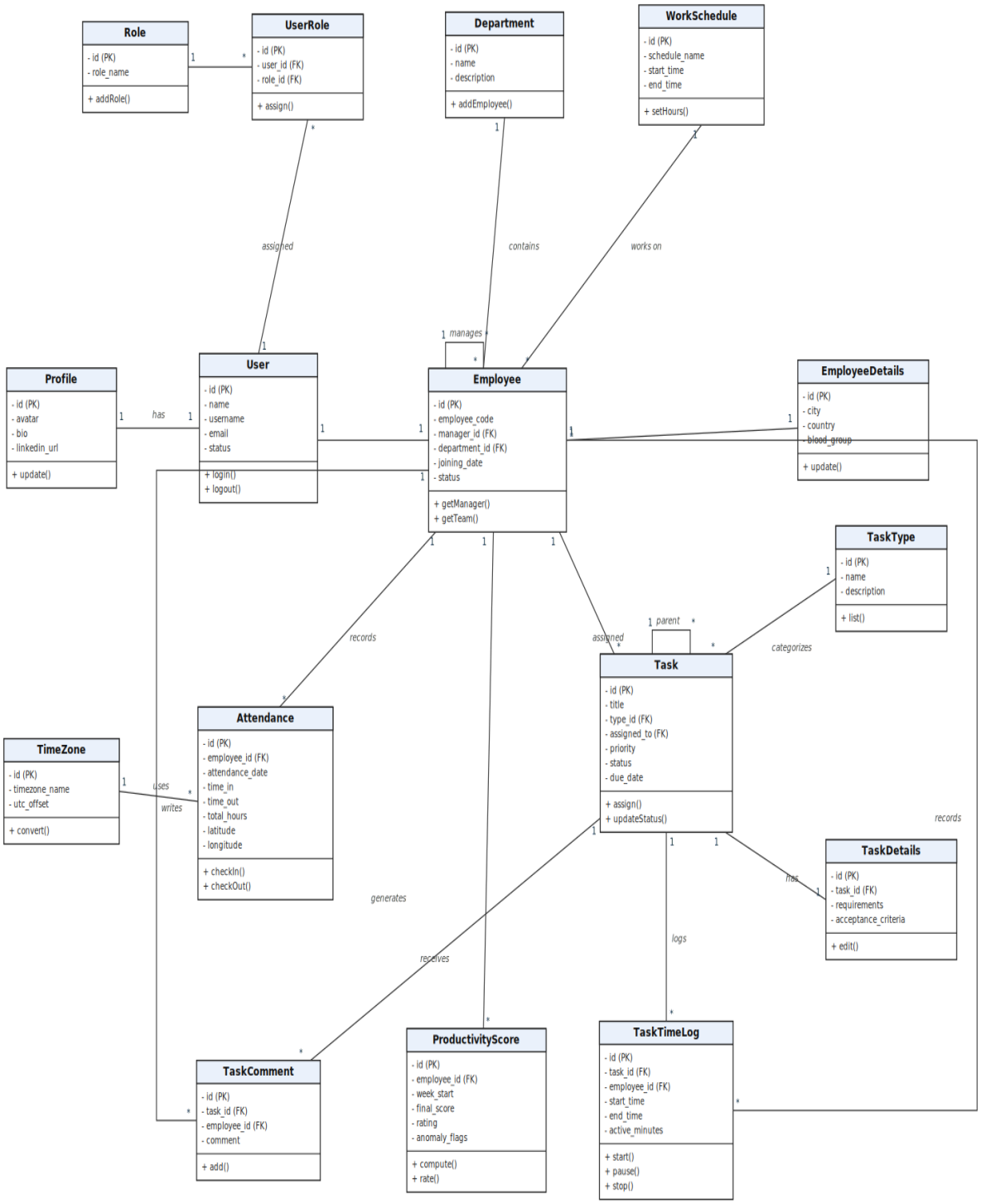

2.1
Receive Check-In Request
Check-In Request
2.2
Capture GPS Coordinates
GPS Coordinates
2.3
Geo-Fence Validation
User ID
Employee
Geo-Fence Result
2.6
Determine Attendance Status
IP Validation Result
2.5
IP Anomaly Detection
IP Address
2.4
Capture Client IP
Status: At Office / Remote / Suspicious
2.7
Store Attendance Record
(Timestamp, GPS, IP, Status)
D1
Attendance Table


P4
AI Productivity Engine
4.1
Fetch Weekly Attendance
4.2
Fetch Task Records
D1
Attendance Table
Weekly Att. Records
Task Records
D2
Tasks Table
Attendance Score 20%
4.3
Calculate Sub-Scores
Sub-Scores
4.4
Compute Final Productivity Score
Task Data
4.7
Trigger Weekly Report Gen.
Report Data
Module
Reporting
Final Score 0-100
4.6
Store Productivity Score
Anomaly Flags, Risk Indicators
4.5
Analyze Behavioral Patterns
Productivity Score, Rating Label
D3
Productivity Score Table

7.2 Entity Relationship Diagram-

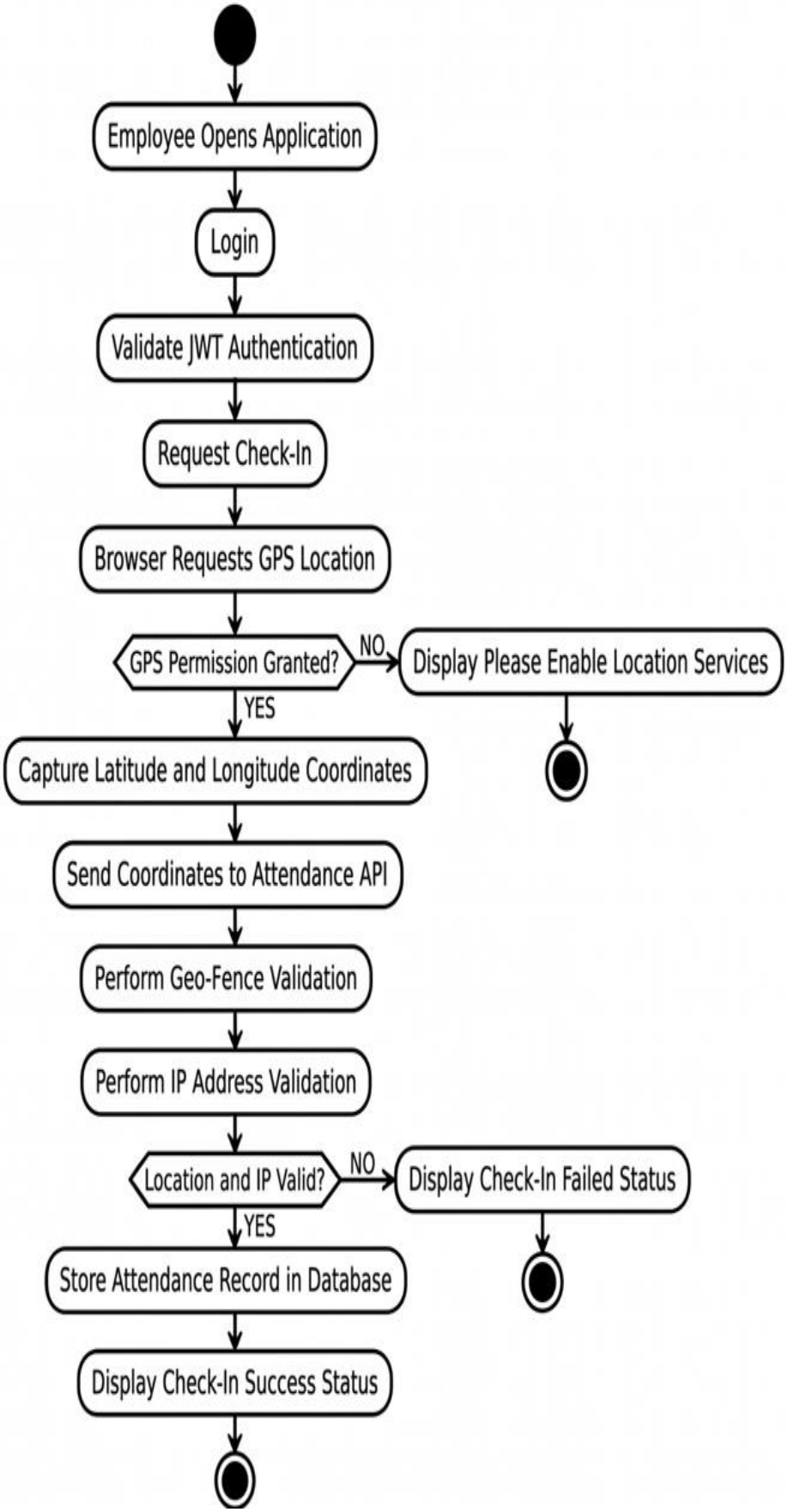


7.3 Class Diagram-

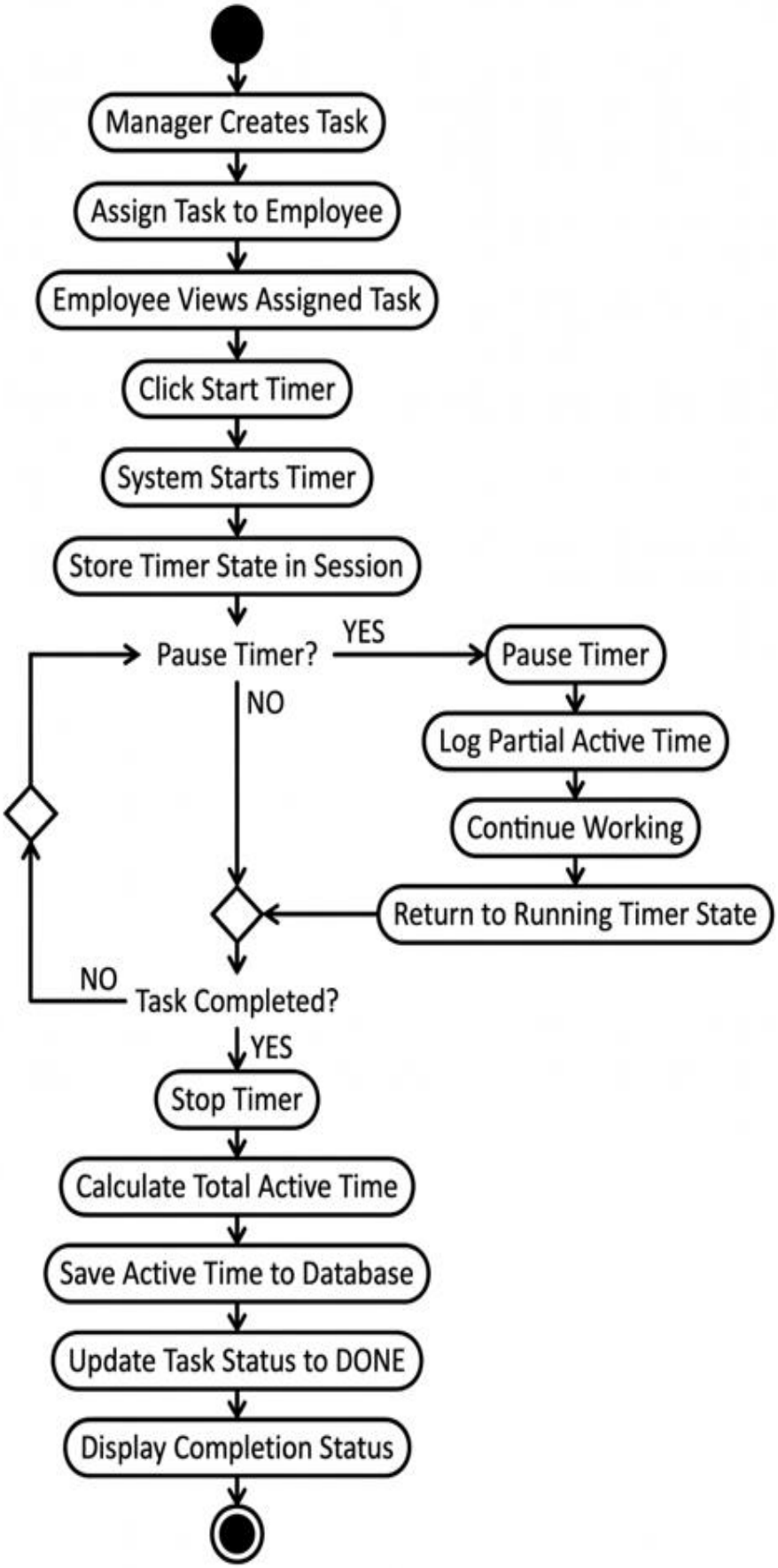


7.4 Activity Diagram-

Employee Attendance Check-In flow-



Task Timer Management Flow-



8. Database And Module Details:

8.1 Database tables-

Table : users

Column Name	Data Type	Constraints
id	BIGSERIAL	PK
name	VARCHAR(150)	NOT NULL
username	VARCHAR(100)	UNIQUE
email	VARCHAR(255)	UNIQUE
password	TEXT	NOT NULL
phone	VARCHAR(20)	
gender	VARCHAR(20)	
dob	DATE	
profile_id	BIGINT	FK → profiles.id
is_verified	BOOLEAN	DEFAULT FALSE
status	VARCHAR(20)	Active, Inactive, Blocked, Password Changed
created_at	TIMESTAMP	
updated_at	TIMESTAMP	
deleted_at	TIMESTAMP	

Table : profiles

Column Name	Data Type	Constraints
id	BIGSERIAL	PK
avatar	TEXT	
bio	TEXT	
address	TEXT	
emergency_contact	VARCHAR(20)	
linkedin_url	VARCHAR(255)	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	
deleted_at	TIMESTAMP	

Table : roles

Column Name	Data Type	Constraints
id	BIGSERIAL	PK
role_name	VARCHAR(50)	UNIQUE
created_at	TIMESTAMP	
updated_at	TIMESTAMP	
deleted_at	TIMESTAMP	

Values: Admin, Manager, Staff

Table : user_roles

Column Name	Data Type	Constraints
id	BIGSERIAL	PK
user_id	BIGINT	FK → users.id
role_id	BIGINT	FK → roles.id
created_at	TIMESTAMP	
updated_at	TIMESTAMP	
deleted_at	TIMESTAMP	

Table : Departments

Column Name	Data Type	Constraints
id	BIGSERIAL	PK

name	VARCHAR(100)	UNIQUE
description	TEXT	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	
deleted_at	TIMESTAMP	

Values: Design, Development, QA, DevOps, HR

Table : work_schedules

Column Name	Data Type	Constraints
id	BIGSERIAL	PK
schedule_name	VARCHAR(100)	
start_time	TIME	
end_time	TIME	
break_minutes	INTEGER	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	
deleted_at	TIMESTAMP	

Table : employee_details

Column Name	Data Type	Constraints
id	BIGSERIAL	PK
address	TEXT	
city	VARCHAR(100)	
state	VARCHAR(100)	
country	VARCHAR(100)	
postal_code	VARCHAR(20)	
blood_group	VARCHAR(10)	
marital_status	VARCHAR(20)	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	
deleted_at	TIMESTAMP	

Table : employees

Column Name	Data Type	Constraints
id	BIGSERIAL	PK
user_id	BIGINT	FK → users.id
employee_code	VARCHAR(50)	UNIQUE
manager_id	BIGINT	FK → employees.id
department_id	BIGINT	FK → departments.id
work_schedule_id	BIGINT	FK → work_schedules.id
employee_details_id	BIGINT	FK → employee_details.id
joining_date	DATE	
status	VARCHAR(20)	Active, Inactive, Blocked
created_at	TIMESTAMP	
updated_at	TIMESTAMP	
deleted_at	TIMESTAMP	

Table : time_zones

Column Name	Data Type	Constraints
id	BIGSERIAL	PK
timezone_name	VARCHAR(100)	
utc_offset	VARCHAR(20)	
created_at	TIMESTAMP	

updated_at	TIMESTAMP	
deleted_at	TIMESTAMP	

Table : attendence

Column Name	Data Type	Constraints
id	BIGSERIAL	PK
employee_id	BIGINT	FK → employees.id
time_zone_id	BIGINT	FK → time_zones.id
attendance_date	DATE	
time_in	TIMESTAMP	
time_out	TIMESTAMP	
total_hours	NUMERIC(5,2)	
latitude	DECIMAL(10,7)	
longitude	DECIMAL(10,7)	
location	TEXT	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	
deleted_at	TIMESTAMP	

Table : task_types

Column Name	Data Type	Constraints
id	BIGSERIAL	PK
name	VARCHAR(100)	UNIQUE
description	TEXT	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	
deleted_at	TIMESTAMP	

Values: Bug Fix, Feature Development, Enhancement, Research, Documentation

Table : task

Column Name	Data Type	Constraints
id	BIGSERIAL	PK
title	VARCHAR(255)	NOT NULL
description	TEXT	
type_id	BIGINT	FK → task_types.id
assigned_to	BIGINT	FK → employees.id
assigned_by	BIGINT	FK → employees.id
parent_task_id	BIGINT	FK → tasks.id
priority	VARCHAR(20)	Low, Medium, High, Critical
impact_type	VARCHAR(50)	
estimated_hours	NUMERIC(5,2)	
due_date	DATE	
status	VARCHAR(20)	WIP, Review, Completed, Failed, Cancelled
created_at	TIMESTAMP	
updated_at	TIMESTAMP	
deleted_at	TIMESTAMP	

Table : task_details

Column Name	Data Type	Constraints
id	BIGSERIAL	PK
task_id	BIGINT	FK → tasks.id

requirements	TEXT	
acceptance_criteria	TEXT	
remarks	TEXT	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	
deleted_at	TIMESTAMP	

Table : task_comments

Column Name	Data Type	Constraints
id	BIGSERIAL	PK
task_id	BIGINT	FK → tasks.id
employee_id	BIGINT	FK → employees.id
comment	TEXT	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	
deleted_at	TIMESTAMP	

Table : task_time_logs

Column Name	Data Type	Constraints
id	BIGSERIAL	PK
task_id	BIGINT	FK → tasks.id
employee_id	BIGINT	FK → employees.id
start_time	TIMESTAMP	
end_time	TIMESTAMP	
active_minutes	INTEGER	
idle_minutes	INTEGER	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	
deleted_at	TIMESTAMP	

Table : productivity_scores

Column Name	Data Type	Constraints
id	BIGSERIAL	PK
employee_id	BIGINT	FK → employees.id
week_start	DATE	
att_score	NUMERIC(5,2)	
task_score	NUMERIC(5,2)	
deadline_score	NUMERIC(5,2)	
active_time_score	NUMERIC(5,2)	
final_score	NUMERIC(5,2)	
rating	VARCHAR(15)	Excellent, Good, Average, Risk
anomaly_flags	JSONB	
generated_at	TIMESTAMP	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	
deleted_at	TIMESTAMP	

8.2 Relationships Table-

Parent Table	Clild Table	Relationship
profiles	users	One-to-One
users	user_roles	One-to-Many
roles	user_roles	One-to-Many

users	employees	One-to-One
departments	employees	One-to-Many
work_schedules	employees	One-to-Many
employee_details	employees	One-to-One
employees	employees	One-to-Many (Self Join)
employees	attendance	One-to-Many
time_zones	attendance	One-to-Many
task_types	task	One-to-Many
employees	task	One-to-Many
task	task	One-to-Many (Self Join)
task	task_details	One-to-One
task	task_comments	One-to-Many
employees	task_comments	One-to-Many
task	task_time_logs	One-to-Many
employees	task_time_logs	One-to-Many
employees	productivity_scores	One-to-Many

8.3 Module Descriptions-

Module	Description	Estimated Effort
M1: Authentication	JWT login, registration, role assignment, password hashing, token refresh	1 week
M2: Attendance	GPS capture, geo-fence comparison, IP validation, check-in/out API, status assignment	2 weeks
M3: Task Management	Task CRUD, assignment, priority/deadline, in-browser timer, status transitions	2 weeks
M4: AI Assessment System	Weighted score formula, rating labels, anomaly pattern detection, weekly Celery trigger	2 weeks
M5: Reporting	PDF report generator, weekly email via SendGrid, admin analytics aggregation	1 week
M6: Access Control Panel	Real-time attendance view, task metrics, employee rankings, attendance heatmap	2 weeks

8.4 Process Logic-

M1 – Authentication Module

- User submits email + password via React form.
- FastAPI endpoint hashes and compares password using bcrypt.
- On match, generate JWT (access + refresh) and return with role payload.
- All subsequent API requests carry Bearer token in Authorization header.
- Role middleware validates token and restricts access per role.

M2 – Attendance Module

- Employee clicks Check-In; browser requests Geolocation permission.
- Coords sent to /attendance/checkin endpoint via HTTPS POST.
- Server computes Haversine distance between employee coords and company geo-fence center.
- If distance ≤ radius: status = 'office'. If approved remote: 'remote'. Else: 'suspicious'.
- IP captured from request header, checked against anomaly rules.
- Record persisted in attendance table.

M3 – Task Management Module

- Manager creates task via dashboard, sets deadline, priority, estimated time, assigns employee.
- Task stored with status = 'pending'.
- Employee views task, clicks Start → timestamp stored in session; status → 'in_progress'.
- On Pause, elapsed seconds added to active_time_sec, timestamp cleared.
- On Complete, final active_time_sec saved; status → 'done'.
- Celery job nightly checks deadline vs. status; marks overdue tasks 'delayed'.

M4 – AI Assessment System

- Celery beat triggers score computation every Sunday at midnight.

- Fetch attendance records for the past 7 days: compute att_score = (days_present / working_days) * 100.
- task_score = (tasks_completed / tasks_assigned) * 100.
- deadline_score = (tasks_on_time / tasks_completed) * 100.
- active_time_score = (active_time_sec / (check_in_to_check_out_sec * 0.7)) * 100, capped at 100.
- final_score = (att_score * 0.20) + (task_score * 0.30) + (deadline_score * 0.20) + (active_time_score * 0.30).
- Assign rating: ≥90 = Excellent, 75–89 = Good, 60–74 = Average, <60 = Risk.
- Detect anomalies: score < 60, 3+ delayed tasks in week, active_time < 30% of shift.
- Persist to productivity_scores, trigger M5 report generation.

M5 – Reporting Module

- Weekly Celery job aggregates scores and anomaly flags for all company employees.
- Generate structured HTML report (converted to PDF via WeasyPrint).
- Email dispatched to admin via SendGrid with report attached.
- Report contents: attendance summary, top performer, at-risk employees, delayed task count, avg score.

8.5 Data Structures-

Object	Key Fields
UserSession	{ user_id, role, company_id, exp, iat } — JWT payload
AttendancePayload	{ user_id, latitude, longitude, ip_address } — API request body
TaskTimerState	{ task_id, start_timestamp, accumulated_seconds } — in-memory session state
ScoreResult	{ user_id, week_start, att_score, task_score, deadline_score, active_time_score, final_score, rating, anomaly_flags }
ReportPayload	{ company_id, week_start, employees: [ScoreResult], top_performer, risk_employees }

8.6 Implementation Methodology

- Agile / iterative development: two-week sprints per module.
- REST API architecture: all client-server communication via versioned JSON endpoints (/api/v1/).
- Modular Python services: each module (auth, attendance, tasks, scoring) is a separate FastAPI router.
- Celery workers run independently, consuming jobs from Redis queue.
- React frontend uses Context API for state management; Axios for API calls.
- Unit tests per module using pytest; frontend component tests via Jest.
- Docker Compose for local development; single-command environment setup.

8.7 Reports Generated-

Report	Trigger	Recipient
Daily Attendance Summary	End of workday (Celery)	Admin
Weekly AI Performance Report	Every Monday 08:00 (Celery beat)	Admin via email
Task Completion Report	On-demand via dashboard	Admin / Manager
Productivity Ranking Report	Weekly, per-company	Admin
At-Risk Employee Alert	When final_score < 60	Admin (email notification)

9. Overall Network Architecture:

TRACE follows a three-tier cloud architecture:

Tier	Component	Technology
Tier 1: Client	Browser (React SPA)	HTTPS over TLS 1.2+
Tier 2: Application	FastAPI + Uvicorn (ASGI)	AWS EC2 / DigitalOcean Droplet
Tier 2b: Background	Celery Workers + Redis Broker	Separate worker process / container
Tier 3: Data	PostgreSQL Database	Managed DB (AWS RDS / DO Managed DB)
External Services	SendGrid (Email), Browser GPS API	HTTPS outbound from app server

All traffic between client and server is encrypted via HTTPS. The AI scoring engine runs as a background Celery worker and does not block the main API. WebSockets provide live updates to admin dashboards without polling.

10. Security Mechanisms:

Level	Mechanism	Details
Transport	Server-side geo-fence validation	GPS coordinates validated server-side; client cannot self-approve location
Authentication	IP anomaly detection	Flag foreign-country login, rapid IP change, shared IP among multiple users
Authorization	Device fingerprinting	Browser fingerprint (OS, browser, screen) stored on first login; mismatch flagged
Geo-location	Password hashing	bcrypt with salt rounds ≥ 12; plain passwords never stored
Network	Encrypted DB storage	Sensitive columns encrypted at rest on managed DB service
Session	Audit logging	All login, check-in, score events timestamped in event log table
Data	Pydantic validation	All API inputs validated server-side via Pydantic schemas
Data	Server-side geo-fence validation	GPS coordinates validated server-side; client cannot self-approve location
Audit	IP anomaly detection	Flag foreign-country login, rapid IP change, shared IP among multiple users
Input	Device fingerprinting	Browser fingerprint (OS, browser, screen) stored on first login; mismatch flagged

11. Future Scope and Further Enhancements:

- Mobile Application: React Native app for one-tap check-in using native GPS.
- Payroll Integration: Connect productivity scores to automated payroll calculation.
- Advanced Burnout Prediction: ML model trained on score trends to predict employee burnout risk.
- Webcam Face Verification: Optional face photo capture at check-in for stronger identity assurance (no hardware required).
- Blockchain Attendance Ledger: Immutable attendance log using a lightweight blockchain for audit-proof records.
- Slack / Teams Integration: Push daily attendance summaries and task alerts into existing communication tools.
- Offline Mode: PWA support with IndexedDB sync for low-connectivity environments.
- Multi-language Support: Internationalization (i18n) for non-English deployments.

12. Bibliography:

- FastAPI Official Documentation – https://fastapi.tiangolo.com
- SQLAlchemy ORM Documentation – https://docs.sqlalchemy.org
- Celery Project Documentation – https://docs.celeryq.dev
- React.js Official Documentation – https://react.dev
- PostgreSQL 15 Documentation – https://www.postgresql.org/docs/
- IEEE Recommended Practice for SRS (IEEE Std 830-1998)
- Haversine Formula – R. W. Sinnott, 'Virtues of the Haversine', Sky and Telescope, 1984
- JWT RFC 7519 – https://datatracker.ietf.org/doc/html/rfc7519
- SendGrid API Documentation – https://docs.sendgrid.com
- scikit-learn Documentation – https://scikit-learn.org/stable/documentation.html
- Docker Documentation – https://docs.docker.com